

---

# DELEGATED AUTHORITY IS AN INPUT, NOT AN IN-FERENCE: PRE-EXECUTION RISK CHECKING FOR TOOL-USING AGENTS

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Guardrail models classify a proposed tool call from its text. For delegated agents, risk has a second component, authorization, whose inputs are the principal’s entitlements, the agent’s delegated scope, and the policy in force. We build a checker whose typed input carries them and use it as a diagnostic. Removing those fields from a trained 2B checker drops macro AUPRC from 0.96 to 0.79 on held-out tools, confined to the three labels that depend on them; a blind LLM judge loses the same labels, and a zero-shot safety model given the same policy text does not use it. Policy checking generalises to unseen phrasings but not unseen policy kinds, and an apparent multi-policy stacking collapse is one fail-closed bundle. The main evidence is external: on AgentDojo and InjecAgent, policy checking transfers zero-shot, while injection detection did not until that failure exposed a label leak in our generator that every synthetic split had scored at its ceiling. Retrained leak-free, the checkers rank the poisoned InjecAgent observation above its clean twin on 98% of pairs; auditing that construction surfaced a length confound we report but could not rule out.

## 1 INTRODUCTION

Deployed AI agents act with delegated authority: a support agent issues refunds for a merchant, a coding agent pushes commits for a developer, an operations agent grants IAM roles for an administrator. Before such an action executes, some system must decide whether it may proceed. The dominant tool is a guardrail classifier that reads the proposed action and returns a harm judgment (Inan et al., 2023; Padhi et al., 2024; Zeng et al., 2024; Ghosh et al., 2024; OpenAI, 2025).

This conflates two questions. **Content risk** asks whether an action looks dangerous on its face and is answerable from the action text. **Authorization risk** asks whether *this* action is something *this* agent was allowed to do, on behalf of *this* principal, under the policy in force. Its inputs are the principal’s entitlements, the agent’s delegated scope, and the applicable clause. Symbolic engines (Pang et al., 2019; Cutler et al., 2024; OASIS, 2013) answer it exactly when the policy is a compiled predicate; the open case is policy written in natural language by a customer, checked against a call its author never saw.

We build a pre-execution risk classifier, *forecheck*, whose typed input contract carries these fields, and use it as an instrument. The identity ablation of §3 is not a discovery: three of the eleven risk labels are functions of the fields we remove, so their collapse is guaranteed. What it measures is how much residual signal the rendered content carries about those labels (almost none on two), whether the other eight dimensions are disturbed (no, but most sit at a ceiling), and whether models that are *given* the context use it. A zero-shot safety model and the acting agent, prompted with the same entitlements and policy text, do not (§3), with the caveat that we cannot fully separate a capability gap from a prompting mismatch. A blind LLM judge shown only the rendered text loses the same two labels when identity is removed (§6).

The main evidence is external (§7). On AgentDojo and InjecAgent, traces we did not generate, policy checking against predicates we wrote transfers zero-shot. Prompt-injection detection did not

| System                        | Reads                     | Authority input                            | Decides                     | Measures |
|-------------------------------|---------------------------|--------------------------------------------|-----------------------------|----------|
| Llama Guard, Granite Guardian | turn, optional call       | none                                       | harm category               | learned  |
| Agent Action Guard            | tool name, arguments      | none                                       | risk score                  | learned  |
| AlignmentCheck, Task Shield   | trace, user goal          | user goal (text)                           | goal alignment              | learned  |
| CaMeL                         | program trace, provenance | data-flow capabilities                     | allow/deny per call         | symbolic |
| Progent                       | call                      | privilege policy (DSL)                     | allow/deny per call         | symbolic |
| Cedar, XACML, Zanzibar        | typed request             | compiled policy, relations                 | allow/deny                  | symbolic |
| forecheck                     | call, trajectory          | entitlements, delegated scope, clause text | 11 calibrated probabilities | learned  |

Table 1: What each system reads, which authority input it takes, and what it decides. forecheck is the only learned checker that takes typed entitlements and delegated scope; the symbolic engines take them but not natural-language clauses. \*Progent’s policies can be LLM-generated; enforcement is symbolic.

transfer at 2B, and tracing that failure exposed a label leak in our own generator: an argument key present in every injected synthetic call and no other. The synthetic metric had been blind to it because a quarter of synthetic positives carry no injected text, capping any text-reading detector near the score the checker reached. Retrained leak-free, the checkers rank the poisoned InjecAgent observation above its clean twin on 98% of pairs. Auditing that construction then surfaced two confounds of the same class, one we rule out from saved scores and one we cannot. We also correct our own earlier reading of a multi-policy composition result (§5). Every number outside §7 is on synthetic data.

## 2 PROBLEM SETTING

**Contract.** forecheck scores a proposed tool call against an `ActionContext`: the principal’s entitlements, the scope delegated to the agent (a possibly strict subset), a trust-labelled trajectory of prior observations, the applicable policy clause(s), and the proposed action (tool, arguments, destination), all as typed fields. It returns one probability per risk dimension. A separate policy engine turns the vector into ALLOW/REVIEW/DENY under a customer-authored bundle; the checker itself never authorises.

**Threat model.** The adversary controls tool outputs and retrieved documents, which the checker renders verbatim inside tag-escaped trajectory blocks so untrusted text cannot impersonate a trusted field. The adversary does not control the entitlement records or the policy store; the thesis depends on that assumption. The checker is itself an injection target, and adaptive attacks against it are not evaluated here.

**Eleven risk dimensions.** Five are decidable from the rendered call alone once the renderer includes the deciding field: `destructive_or_irreversible_action`, `financial_commitment`, `external_communication`, `untrusted_destination`, and, from generator version v4, `privilege_escalation`, whose authority-before/after fields are written into the call text. `sensitive_data_exposure` depends on a destination fact the renderer omits (§6). Two are trajectory dimensions (`prompt_injection_influence`, `suspicious_action_sequence`). Three are functions of the identity and policy fields: `unauthorized_scope` (is the required scope inside the principal’s entitlements and the delegated scope?), `policy_conflict` (does the call violate a stated clause?), and `insufficient_context` (was enough delegated information present to decide?). Whether a reader of the text can recover each label is tested in §6.

**Data.** A template generator samples a latent scenario, derives the eleven labels deterministically from it, and renders a context. Table 2 lists the versions used; where a comparison depends on their differences we say so. Known defects beyond the two the generator was revised for: a 5% chance of marking any resource irreversible, including reads; a 5% chance of a financial amount on a non-financial tool; and a background destination on 15–35% of non-sending calls, which drives the `untrusted_destination` and `sensitive_data_exposure` labels without appearing in the call. Splits are `train`, `calibration`, `dev`, `test`, `heldout_family` (a stratified subset of

| Version | Change from the previous version                                                                                                                                                        |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v2      | Baseline; <code>privilege_escalation</code> deciding fields not rendered                                                                                                                |
| v4      | Renders <code>authority-before/after</code> into the call; 15 policy kinds                                                                                                              |
| v5      | 30 policy kinds (4 withheld in the kind split)                                                                                                                                          |
| v6      | Removes the injection label leak (§7); fills the target argument for benign calls too; 300–2,000 character multi-format observations with the injected instruction at a random position |

Table 2: Generator versions. v3 is v2 with the policy splits and is reported only in the appendix. Row counts are 49,996 for v2 and v6.

tools withheld), and the two policy splits of §4; `heldout_family` scores within 0.01 of test for every arm. Every synthetic number measures whether an arm learned the generator’s label function.

**Arms.** A LoRA-adapted decoder (Hu et al., 2021) at 2B (`granite-3.3-2b-instruct`) and 8B, scored by candidate-token logits per dimension and temperature-scaled on the calibration split (Guo et al., 2017); two linear-head encoders (ModernBERT-large (Warner et al., 2024), `granite-embedding-r2`); a deterministic *rule baseline* of field lookups, whose `policy_conflict` rule is a constant and whose `unauthorized_scope` rule does not intersect entitlements with delegated scope, so on those two dimensions it is an uninformative floor at the positive rate; a *symbolic baseline* that compiles the seen policy phrasings into predicates over the typed fields, intersects entitlements with delegated scope, and abstains (0.5) on clauses it cannot parse; two zero-shot safety models given the same policy text as a custom criterion, `gpt-oss-safeguard 20B` (OpenAI, 2025), which emits a binary verdict, and Granite Guardian 3.3 8B (Padhi et al., 2024), scored from its yes/no verdict logits (§3); and an *agent self-judgment* arm in which the untrained 8B instruct model (IBM Granite Team, 2024) receives the same context in an agent system prompt and returns one ALLOW/STOP verdict.

**Metrics.** We report AUPRC, whose chance level is the positive rate, with per-dimension evaluable  $n$  and positive rate (287 to 6,653 rows; 0.07 to 0.52). Seven dimensions are above 0.99 for the trained checker and four are solved by the rule baseline, so we also report a *non-saturated macro* over the seven dimensions on which the rule baseline is below 0.9. Synthetic estimates are single-seed unless stated; external cells with saved per-example scores carry 95% bootstrap intervals. Hyperparameters and split construction are in Appendix I.

### 3 IDENTITY ABLATION AS A DIAGNOSTIC

At evaluation time only, we remove principal entitlements, delegated scope, `on_behalf_of`, and policy text from the trained decoder’s rendered context (`-strip-identity`), leaving action, destination, and trajectory untouched. Three labels are deterministic functions of the removed fields, so their collapse is expected; the ablation is informative about (i) how much the remaining content predicts them, (ii) whether anything else moves, and (iii) whether arms that keep the context use it. We compare against the rule baseline, a decoder *trained* on identity-free renderings, the same ablation at 8B, and the zero-shot arms (Table 3, Figure 1; v2 and all dimensions in Appendix A).

**Residual content signal is small on two of the three dimensions.** Stripped, `unauthorized_scope` sits at 0.125 against a 0.108 positive rate and `policy_conflict` at 0.392 against 0.371: without the entitlements or the clause, the call text carries almost nothing about either. `insufficient_context` retains 0.412 against 0.087, well above chance; a blind reader cannot recover this label from the text at all (§6), so the residual is a generator regularity. The symbolic baseline scores 1.000 on `unauthorized_scope` and 0.98 on `privilege_escalation` from the same typed fields, so on these two the checker adds nothing over a compiled predicate; its case rests on `policy_conflict` and on inputs a compiler cannot parse.

| Arm                                       | Full  | Stripped |
|-------------------------------------------|-------|----------|
| Decoder 2B v4                             | 0.960 | 0.790    |
| non-saturated macro (7 dims)              | 0.939 | 0.672    |
| Decoder 8B v4                             | 0.958 | 0.786    |
| Decoder 2B v4, trained stripped           | —     | 0.784    |
| Decoder 2B v2                             | 0.915 | 0.736    |
| Encoder ModernBERT-large v2               | 0.813 | 0.712    |
| Rule baseline                             | 0.614 | 0.614    |
| non-saturated macro (7 dims)              | 0.404 | 0.404    |
| Guardian 3.3 8B, 0-shot ( $n=2000$ )      | 0.419 | 0.407    |
| gpt-oss-safeguard 20B, binary ( $n=400$ ) | 0.277 | 0.270    |
| Agent self-judgment 8B ( $n=2000$ )       | 0.242 | 0.281    |

Table 3: Macro AUPRC over eleven dimensions on `heldout_family`, with and without identity/policy context. `gpt-oss-safeguard` emits a 0/1 verdict, so its AUPRC is a single operating point and is not comparable to the continuous-score rows.

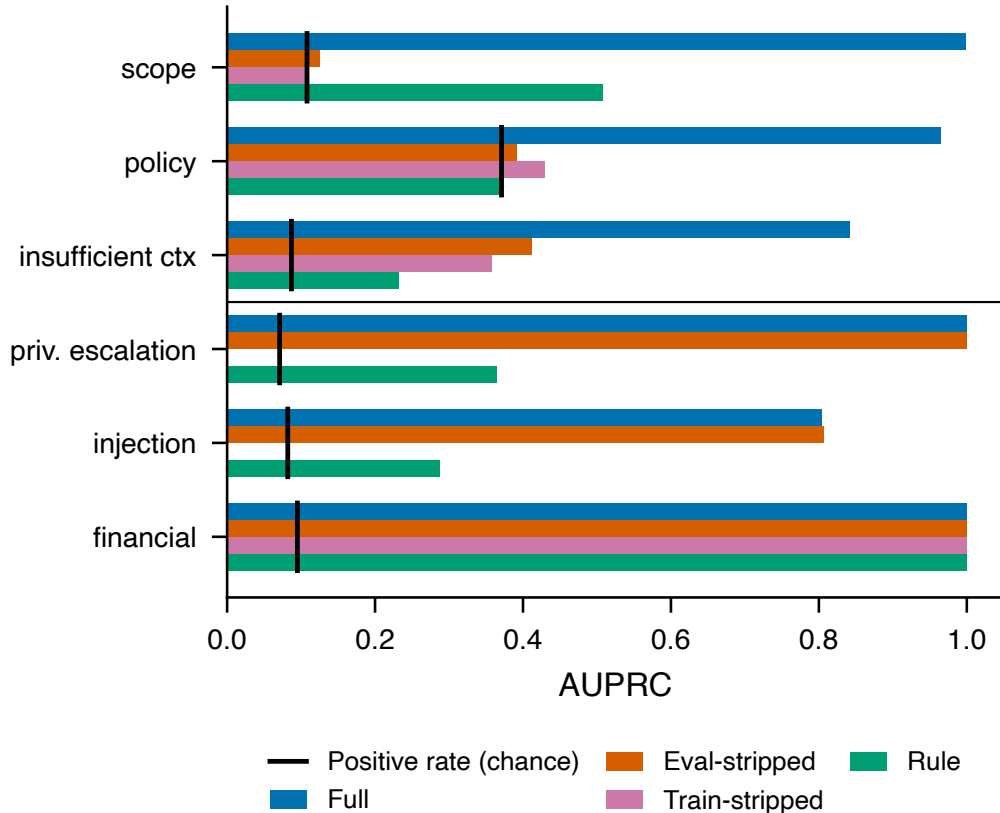


Figure 1: Decoder 2B v4, `heldout_family`: AUPRC per dimension with full input, identity removed at evaluation, identity removed at training, and the rule baseline; black ticks mark each dimension’s positive rate (chance). The top three dimensions are functions of the removed fields. v4 injection text carries the label leak of §7; its evidence sits in the call arguments, which stripping does not touch. Evaluable  $n$  and exact values in Table 8.

**Nothing else moves, but most of it could not.** The eight remaining dimensions change by at most 0.002. Five are at 1.000 for the checker, three of those for the rule baseline too, so “content dimensions are unaffected” is largely a ceiling effect. `privilege_escalation` is unaffected because the v4 renderer writes the deciding authority fields into the call (before v4 it scored 0.46–

0.53); the v4 injection score is unaffected because its evidence was a leaked argument key in the call (§7).

**Training without the fields gives the same ceiling.** A decoder trained on identity-free renderings reaches macro 0.784 against 0.790 for eval-time stripping, within 0.06 per dimension, so the drop is not a broken shortcut or a prompt-format shift; 8B collapses identically (0.958 to 0.786).

**gpt-oss-safeguard is at chance everywhere; Guardian reads the fields weakly.** Given the same policy text as a custom criterion, gpt-oss-safeguard scores 0.277 macro, below the rule baseline’s 0.614, and within 0.09 of the positive rate on ten of eleven dimensions, including content dimensions the rule baseline solves exactly (Table 9); the exception is `policy_conflict` (0.49 against 0.29,  $n=103$ ). The corrected Guardian read is not at chance here but is weak: `unauthorized_scope` is 0.171 AUPRC, 0.706 AUROC with the fields, falling to 0.104 AUPRC, 0.488 AUROC (chance, at its 0.106 positive rate) stripped, against the trained checker’s 0.999; `policy_conflict` goes 0.477/0.610 to 0.420/0.555 against a 0.385 base rate and the checker’s 0.964; `insufficient_context` stays at 0.085 AUPRC, 0.53 AUROC either way, against 0.079, so the fields add nothing there. Neither zero-shot model separates “lacks authorization reasoning” from “this prompt does not reach it.”

**Neither does the acting agent.** Asked inside its own system prompt, with the same entitlements and policy text, whether to proceed, the untrained 8B model returns one ALLOW/STOP verdict that is at chance on the three identity dimensions (AUROC 0.54–0.67; macro 0.242 against the checker’s 0.960 on the same 2,000 rows). On InjecAgent the same prompt scores 1.000 on that benchmark’s scope label, because there “out of scope” means “nobody asked for this”, which an agent can see; checking a call against a delegated grant is a different question (Appendix E).

## 4 POLICY GENERALISATION: PHRASING VS. KIND

Robustness to unseen *tools* is not robustness to unseen *policies*, and customers write their own clauses. `heldout_policy_kind` withholds entire predicate kinds (e.g. `forbid_recipient_domain`) from training; `heldout_policy_phrasing` withholds one paraphrase of every kind the model did train on. v4 has 15 kinds with 2 withheld; v5 has 30 with 4.

**Unseen phrasing generalises; unseen kinds do not.** A paraphrase never seen in training leaves `policy_conflict` at 0.98–0.99, matching the test split: the model reads the clause’s meaning, not a template id. On withheld kinds 2B lands at 0.717 (v4) and 0.725 (v5), well above the positive-rate floor but far from the phrasing ceiling. 8B reaches 0.838 on two seeds. The compiled-predicate baseline sits at 0.90–0.96 wherever the phrasing inventory is known and falls to 0.600 on withheld kinds, so on this generator the learned checker matches a symbolic compiler on seen kinds and beats it by 0.1–0.2 on unseen ones; neither reads a new kind reliably. The supportable claim is “one model, policies of a kind seen in training, any phrasing,” not “arbitrary new policy kinds.”

**What the kind split can and cannot say.** The v4 and v5 kind cells are means over training seeds on *different* withheld kinds (2 vs. 4), so their near-equality is not evidence that a larger inventory fails to help: the dominant variance component, which kinds were withheld, is not estimated, and with three seeds at sd 0.03 the minimum detectable difference is about 0.08. We report “no evidence of an inventory effect at this size,” not its absence.

| Split                                               | 2B v4                         | 2B v5        | 8B v4        |
|-----------------------------------------------------|-------------------------------|--------------|--------------|
| test                                                | 0.994                         | 0.996        | 0.990        |
| heldout_family                                      | 0.964                         | 0.933        | 0.965        |
| heldout_policy_phrasing                             | 0.993                         | 0.980        | 0.978        |
| heldout_policy_kind                                 | <b>0.717</b>                  | <b>0.725</b> | <b>0.838</b> |
| rule baseline (= pos. rate)                         | 0.405                         | 0.325        | 0.405        |
| symbolic baseline*: test / family / phrasing / kind | 0.947 / 0.900 / 0.956 / 0.600 |              |              |

Table 4: `policy_conflict` AUPRC across splits. Kind-split cells are means over training seeds (2B: three seeds, sd 0.032 for v4 and 0.026 for v5; 8B: two seeds, 0.851 and 0.825); other cells are seed 0. The rule baseline’s policy rule is a constant, so it scores the positive rate. \*Compiled predicates over typed fields, run on splits regenerated with the current 30-kind inventory (v5/v6 kinds, 4 withheld); it abstains on 67% of kind-split clauses and on 10–23% elsewhere (kinds whose deciding fact is only in trajectory text). Macro AUPRC and per-seed values in Appendix B.

## 5 MULTI-POLICY COMPOSITION, CORRECTED

Guardrail deployments run several policies and act on the most restrictive decision. We stack  $k$  bundles over the v2 decoder’s probability vector on `heldout_family` ( $n=6640$ ) two ways: independent evaluates each bundle alone and ORs the decisions; `expected_cost` merges the bundles’ rules and decides once by the argmin of expected cost over the joint vector under a fixed cost table (Figure 2; Appendix C). FPR counts any non-ALLOW decision on a covered benign example, so it includes REVIEW. An earlier draft read these numbers as “threshold stacking compounds to a 0.97 false positive rate.” It does not.

**The  $k=3$  jump is one fail-closed bundle.** FPR goes 0.015, 0.140, 0.966 and stays there through fifteen bundles. The third bundle, conservative, has `default_decision: review: alone` it routes essentially every covered call to a human, so the OR saturates when it enters and nothing added afterwards changes it; independent guards at the observed  $k=1$  rates would compound to about 0.04. One fail-closed policy dominates an OR-stack, which XACML’s `deny-overrides` and Cedar’s forbid-trumps-permit semantics (OASIS, 2013; Cutler et al., 2024) make explicit. `expected_cost` is constant from  $k=2$  because the merged rule set already covers every dimension the stack will ever cover. Its level differs by arm (decoder 0.082 vs. encoder 0.921), but the arms also differ in discrimination (granite-r2 0.433 on `policy_conflict` vs. 0.964), so we do not attribute that gap to calibration. The approval-elimination curve on the same probabilities is in Appendix D.

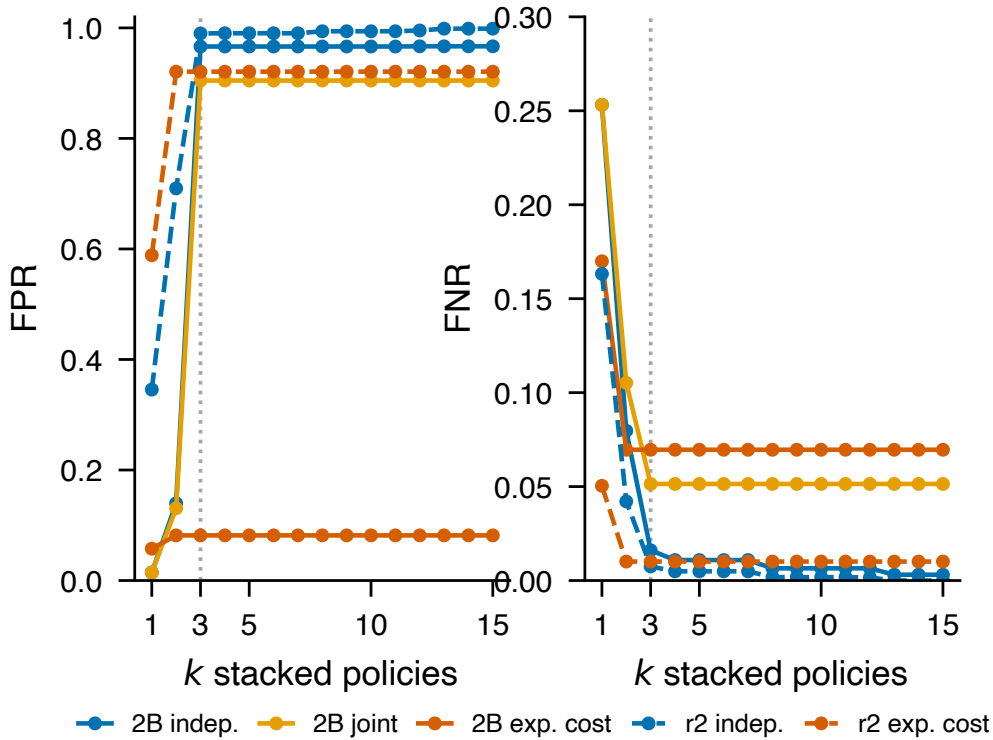


Figure 2: Stacking  $k$  bundles (permissive, balanced, conservative, then eleven single-dimension policies) on `v2 heldout_family`: FPR (REVIEW counts as a false positive) and FNR against  $k$  for the 2B decoder and the granite-r2 encoder under each strategy. The vertical line marks where the fail-closed conservative bundle enters. Values in Table 11.

## 6 ARE THE LABELS OBSERVABLE? A BLIND JUDGE

Is each label recoverable from the rendered text by a reader who never sees the generator’s latent scenario? A frontier model (gpt-5.6-sol, one prompt, low reasoning effort), given only the rendered context and the eleven dimension definitions, judged 600 `v4` rows stratified to at least 20 positives and 20 negatives per dimension, once with the full rendering and once identity-stripped. The definitions are the generator’s own docstrings, so this tests observability under a shared definition. Agreement with the generator is not ground truth, but low agreement is informative: the label then depends on information the generator never rendered, and a model scoring well on it has learned a generator regularity.  $\kappa$  is on 600 rows without intervals.

**Three tiers.** *Recoverable* ( $\kappa > 0.7$ ): `privilege_escalation`, `policy_conflict` (judge recall 0.89; disagreements are the judge reading a clause more strictly), and, on leaked text, injection. *Definitional disagreement* ( $0.2 \leq \kappa \leq 0.5$ ): six dimensions read two coherent ways; the judge flags twice as many `unauthorized_scope` rows because it counts a mismatch with the agent’s stated objective as out of scope, while the generator’s label is entitlement-based. *Not recoverable* ( $\kappa < 0.1$ ): `insufficient_context` (the judge calls the context sufficient on 50 of 58 generator-positive rows) and `sensitive_data_exposure`, whose label requires the destination to leave the tenant, a fact the rendering does not show; this contradicts §2’s classification of it as call-decidable.

**The judge’s labels move the same way as the checker’s scores.** Stripping identity from the judge’s input drops `policy_conflict`  $\kappa$  from 0.714 to 0.029 (28 judge-positives instead of 259: no clause left to check) and `unauthorized_scope` from 0.391 to 0.040 (203 positives at precision 0.16). Every other dimension is unchanged within noise, and the content dimensions’ judge-yes

| Dimension                               | $\kappa$ full | $\kappa$ stripped | Checker vs. judge | Rule vs. judge |
|-----------------------------------------|---------------|-------------------|-------------------|----------------|
| privilege_escalation                    | 0.909         | 0.909             | 0.977             | 0.297          |
| prompt_injection_influence <sup>†</sup> | 0.905         | 0.905             | 1.000             | 0.459          |
| policy_conflict                         | 0.714         | <b>0.029</b>      | 0.889             | 0.502          |
| unauthorized_scope                      | 0.391         | <b>0.040</b>      | 0.592             | 0.509          |
| destructive_or_irreversible_action      | 0.511         | 0.542             | 0.635             | 0.509          |
| untrusted_destination                   | 0.213         | 0.205             | 0.864             | 0.785          |
| insufficient_context                    | 0.059         | −0.036            | 0.090             | 0.090          |
| sensitive_data_exposure                 | 0.002         | 0.061             | 0.405             | 0.291          |
| macro, 11 dims                          | 0.436         | 0.340             | 0.613             | 0.415          |
| macro, 10 dims (injection excluded)     | 0.389         | 0.284             | 0.574             | 0.411          |

Table 5: Blind-judge agreement with the generator label (yes vs. not-yes, Cohen’s  $\kappa$ ),  $n=600$  v4 rows, full and identity-stripped renderings. Right: AUPRC of the 2B v4 checker and the rule baseline *against the judge’s labels* on the judged `heldout_family` rows ( $n=257$ ). <sup>†</sup>Measured on leaked v4 text; judge and generator could agree by reading the leaked argument, so the 10-dimension macros exclude it. Full table in Appendix F.

counts are identical between passes. These are the two dimensions on which the checker’s residual signal was nil (Figure 1).

**Consequence.** The ablation is interpretable on `policy_conflict`, directionally on `unauthorized_scope`, and not on `insufficient_context`: the checker’s 0.841 there is a generator regularity, and we flag every number on that dimension. Scored against the judge’s labels rather than the generator’s, the checker keeps the recoverable dimensions and beats the rule baseline on each by 0.39–0.68 AUPRC.

## 7 TRANSFER TO DATA WE DID NOT GENERATE

We score the checkers zero-shot on two published benchmarks. Nothing was trained or tuned on either; calibration was fitted on synthetic data and threshold selection is disabled. The leak-free v6 checkers are the paper’s models; v4 rows are the pre-fix comparison.

**AgentDojo** (Debenedetti et al., 2024): banking-suite traces for two frontier agents, 1,714 runs, 4,215 proposed calls, 11 attack types plus no-attack runs. Labels come from AgentDojo’s own `ground_truth()` functions: `prompt_injection_influence` is `yes` when a call executes the injection task’s goal (positive rate 0.150); `unauthorized_scope` when the call’s function is outside the user task’s reference solution (0.253). `policy_conflict` (0.237) is *ours*: four banking policies, each a deterministic predicate over user-named recipients. **InjecAgent** (Zhan et al., 2024): 1,054 cases over 38 toolkits. AgentDojo’s injection positives differ from negatives in the call itself, so it cannot tell whether a checker reads the untrusted text or the call. InjecAgent can: for each attacker tool we build a *poisoned/clean pair*, the same attacker call with identical arguments, proposed after the tool response with the attacker’s instruction filled in (`yes`) or removed (`no`). 4,250 examples, 1,598 pairs; `policy_conflict` from five generic policies about the call’s shape.

**Policy checking transfers, and on AgentDojo it stays identity-dependent.** On banking `policy_conflict`, whose predicates reference recipients the user named, 2B v4 scores 0.849 against a 0.237 base rate on policies, a tool schema, and traces it never saw; eval-time stripping drops it to 0.694 and the train-stripped model to 0.390. The v6 checkers score lower here (0.670, 0.785) with the same stripping pattern (0.521, 0.450). This is the one external result that bears on the identity thesis. On InjecAgent the same dimension scores 0.92–0.98 and stripping barely moves it: the five policies are rules about the call’s shape, so it is a content result.

**Injection before the fix: the v4 heads barely read the untrusted text.** 2B v4 is below base rate on AgentDojo injection (0.128, AUROC 0.348): its injection head never fires on this domain (raw logits for all 4,133 calls in  $[-5, -3]$ ) while its policy head separates the same rows; 8B v4 reaches 0.701. On InjecAgent, with the call held fixed, both v4 heads sit near raw logit  $-4$  on poisoned and clean alike and return an identical score on 24% (2B) and 39% (8B) of pairs (Figure 3). Counting

| Arm                     | AgentDojo banking |             |               | InjecAgent   |             |               | tie-adj. win vs. |              |              |
|-------------------------|-------------------|-------------|---------------|--------------|-------------|---------------|------------------|--------------|--------------|
|                         | inj. (.15)        | scope (.25) | policy* (.24) | inj. (.50)   | scope (.75) | policy* (.46) | clean            | padded       | instr.       |
| Rule baseline           | 0.250             | 0.253       | 0.237         | 0.500        | 0.752       | 0.464         | –                | –            | –            |
| Decoder 2B v6           | 0.693             | 0.214       | 0.670         | 0.878        | 0.704       | 0.922         | 0.980            | 0.986        | 0.989        |
| eval-stripped           | 0.669             | 0.306       | 0.521         | –            | –           | –             | –                | –            | –            |
| Decoder 8B v6           | <b>0.838</b>      | 0.239       | 0.785         | 0.943        | 0.677       | 0.965         | 0.983            | 0.981        | <b>0.993</b> |
| eval-stripped           | 0.819             | 0.354       | 0.450         | <b>0.953</b> | 0.917       | 0.963         | <b>0.986</b>     | <b>0.986</b> | <b>0.993</b> |
| Decoder 2B v4 (pre-fix) | 0.128             | 0.257       | <b>0.849</b>  | 0.549        | 0.732       | 0.965         | 0.624            | 0.514        | 0.515        |
| Decoder 8B v4 (pre-fix) | 0.701             | 0.355       | 0.626         | 0.544        | 0.849       | <b>0.977</b>  | –                | –            | –            |
| eval-stripped           | 0.743             | 0.266       | 0.404         | 0.556        | 0.843       | 0.968         | –                | –            | –            |

Table 6: Zero-shot AUPRC on AgentDojo banking ( $n=4,215$ ; 4,133 for injection) and InjecAgent ( $n=4,250$ ; 3,196 for injection), positive rates in the header; \*labels from predicates we wrote. Right block: tie-adjusted injection win over 1,598 InjecAgent groups against three length-matched controls (clean, padded, instruction; defined in Appendix G), 0.5 is chance; 8B v4 was not scored against them. Full counts and sign tests in Table 19. Agent-self, Guardian and the v4 stripped AgentDojo rows are in Appendix G. The 8B v4 AgentDojo injection cell was 0.797 in an earlier run we could not reproduce; two later runs gave 0.701.

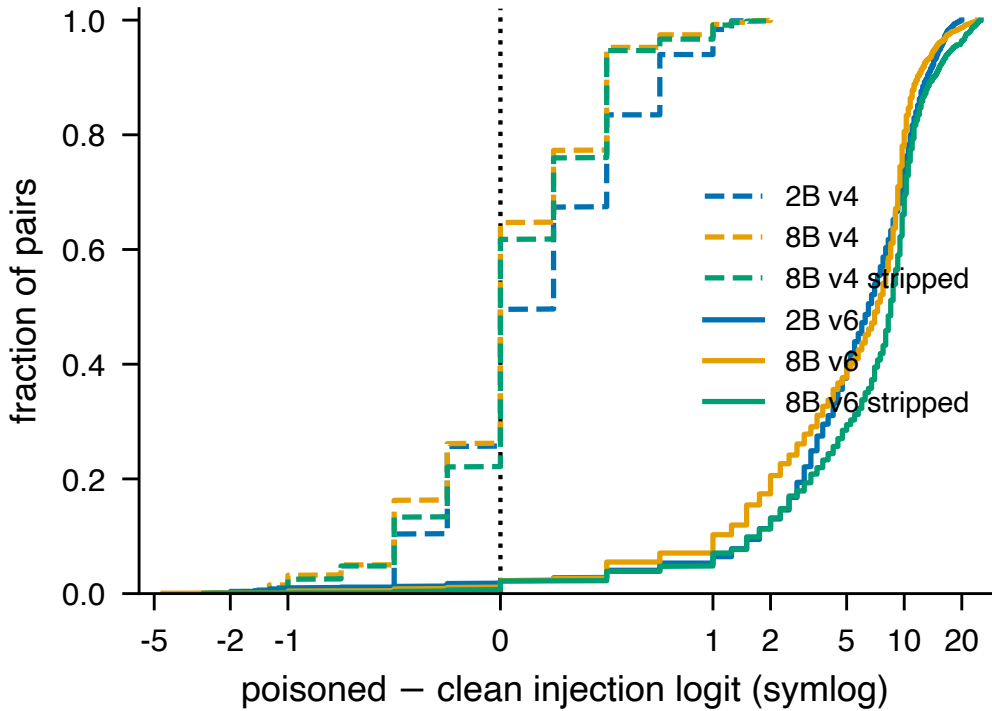


Figure 3: InjecAgent pairs: empirical CDF of the poisoned-minus-clean injection logit over the 1,598 pairs, per checker. The v4 heads (dashed) put a quarter to two fifths of pairs exactly at zero and a further quarter below it; the v6 heads (solid) rank the poisoned variant higher on 98% of pairs. Counts and intervals in Table 18.

ties as half, they rank the poisoned variant higher on 0.624 and 0.546 of pairs, significantly above 0.5 (sign test on decided pairs,  $p < 10^{-5}$ ) but small. Whether the 8B AgentDojo transfer was call-level reading or a head that only fires on long observations is undetermined; a payload-deletion counterfactual with the call held fixed would decide it and has not been run.

**The cause was a label leak in our generator.** Contrasting rendered calls between injected and non-injected synthetic examples showed that the v1–v5 generator wrote an `instructed_target` argument into every injected call and no other (presence 1.00 vs. 0.00 over 13,312 v4 rows) and drew its value from five literals, three containing “attacker” (Appendix H). The synthetic metric could not show this. A quarter of synthetic injection positives come from a compromised tool schema and render no injected text at all, so a detector that reads only the observation and a detector that reads only the leaked key are both capped near 0.75–0.80 AUPRC; the checker’s 0.80 is consistent with either. The synthetic score measured the ceiling, not what the head read.

**v6 restores transfer; the fix is a bundle.** v6 removes the key, fills the target argument for benign calls too, and lengthens and randomises observations. Retrained on v6 with identical size and recipe, 2B reaches 0.693 on AgentDojo injection (AUROC 0.928) and 8B 0.838 (0.966), and both heads fire (2B v6 median positive logit +4.5 vs. −3.0). On InjecAgent both rank the poisoned variant strictly above its clean twin on 0.977 of pairs with under 2% ties; the synthetic injection score did not change (0.804 to 0.787 on `heldout_family`). Observation length alone could explain the AgentDojo gain (2,000-character records against v4’s 132-character templates); the changes were not separated and we claim only that the bundle is sufficient.

**The length confound is tested and closed.** The clean twin is shorter than the poisoned observation in all 1,598 groups; two more controls, padded and instruction, refill the slot with byte-matched benign text instead, shorter than or equal to it in 95.7% of groups. Against both, v6 still wins 0.98–0.99 tie-adjusted (Table 6), while v4’s 0.624 win against plain clean falls to 0.514 against padded, chance ( $p = 0.16$ ): v4’s signal was length, not injection reading. A second confound, destination trust, flips in 39% of groups but does not carry the result: restricted to the 972 groups where it is equal, every v6 win moves by at most 0.015 (Appendix G).

**What does not transfer.** `unauthorized_scope` is near base rate on AgentDojo for every checker (0.21–0.36 vs. 0.253); the label counts harmless auxiliary reads as out of scope. On InjecAgent, 8B v6 is below base rate (0.677 vs. 0.752) until the port’s placeholder principal and scope fields are removed (0.917). Removing identity *helps* on several external cells, which fits the thesis only in its narrow form: the right fields help, generic ones hurt. Calibration does not transfer (macro ECE 0.20–0.27 on AgentDojo vs. under 0.03 synthetic); isotonic refitting on 250–500 labelled calls restores ECE 0.01 with AUPRC unchanged. Over all four AgentDojo suites (30,143 calls), 8B v4 injection holds on three and is weak on travel; 2B v4 is at base rate everywhere; v6 has not been run on that export (Appendix G).

## 8 RELATED WORK

**LLM guardrails.** Llama Guard (Inan et al., 2023), Granite Guardian (Padhi et al., 2024), Shield-Gemma (Zeng et al., 2024), NVIDIA Aegis (Ghosh et al., 2024), and gpt-oss-safeguard (OpenAI, 2025) classify content or harm per turn or action; Agent Action Guard (Pro-GenAI, 2025) is an embedding MLP over tool name and arguments. LlamaFirewall’s AlignmentCheck (Chennabasappa et al., 2025) and Task Shield (Jia et al., 2024) judge alignment with the user’s stated goal, the closest analogues of our `unauthorized_scope`; neither takes a typed entitlement or delegated scope as input.

**Authorization for agents.** Symbolic engines decide allow/deny from a fully specified request and compiled policy: OPA (Open Policy Agent), Cedar (Cutler et al., 2024), XACML with its combining algorithms (OASIS, 2013), and Zanzibar (Pang et al., 2019), whose “is this delegated scope inside the principal’s grant” check is exactly our `unauthorized_scope`. Agent-specific designs enforce authorization by construction: CaMeL (Debenedetti et al., 2025) tracks capabilities through a quarantined LLM’s dataflow, Progent (Shi et al., 2025) applies per-call privilege policies, IsolateGPT (Wu et al., 2025) puts agent applications behind a permission layer, and RTBAS (Zhong et al., 2025) and AirGapAgent (Bagdasarian et al., 2024) enforce principal-dependent information-flow norms. OWASP’s agentic guidance (OWASP GenAI Security Project, 2025) holds that authorization must be deterministic and auditable. Table 1 sets these side by side. Our delta is narrow and compatible with all of these: where the mandate is a natural-language clause nobody has compiled, a learned reader generalises across paraphrases (§4) and emits a probability such an engine can consume; the deterministic check on typed fields remains the right tool for scope.

---

**Prompt-injection defences and benchmarks.** StruQ, SecAlign, and spotlighting (Chen et al., 2024a;b; Hines et al., 2024) harden the acting model; our injection head is a detector in front of it. AgentHarm, ToolEmu, InjecAgent, AgentDojo, and Agent Security Bench (Andriushchenko et al., 2024; Ruan et al., 2023; Zhan et al., 2024; Debenedetti et al., 2024; Zhang et al., 2025) measure end-to-end agent behaviour with an outcome judge; we reuse two as external test sets by deriving per-action labels from their ground-truth functions.

## 9 CONCLUSION

Delegated-authority labels are functions of fields a content classifier never sees; the ablation confirms the content carries almost nothing about two of the three, as expected. What was not guaranteed: models given the context do not use it under a straightforward prompt, a blind reader loses the same labels without the fields, policy checking generalises to unseen wording but not unseen kinds, and a composition collapse was one fail-closed bundle. The strongest result is methodological: a benchmark we did not generate exposed a label leak every synthetic split had scored at its ceiling, and auditing that construction surfaced two more.

## LIMITATIONS

**Synthetic data with enumerated defects.** Every number outside §7 is on data from our own generator and measures whether an arm learned the generator’s label function. No human-labelled or production-traffic evaluation exists and no real-world safety claim is made. Beyond the two defects the generator was revised for, it marks 5% of resources irreversible at random including on reads, stamps a financial amount on 5% of non-financial tools, and attaches a background destination to 15–35% of non-sending calls; `insufficient_context` is not recoverable from the text by a blind reader, and `sensitive_data_exposure` depends on a tenancy fact the rendering does not show. Every number on those two dimensions, and every 11-dimension macro that includes them, carries that caveat. Generator seed nondeterminism was found and fixed mid-project; v2/v3 data are not byte-reproducible from their configs.

**The injection leak contaminates more than §7.** Every synthetic injection number before v6, the judge’s  $\kappa=0.905$  on that dimension, and the agent-self injection reversal in §3 were measured on text carrying the leaked argument. We mark them and report leak-excluded macros, but the judge and the agent-self arm have not been re-run on v6 text. The v6 fix changes observation length and format as well as removing the key; the two are not separated.

**The InjecAgent pair test’s confounds are controlled, not eliminated in general.** The clean twin is shorter than the poisoned one in every pair, and length- and destination-trust-matched controls confirm the v6 win is not either (§7); both controls are themselves synthetic filler text, not real benign tool output, so a checker specialised to distinguish our filler from genuine content would not be caught by them.

**Uncertainty.** Synthetic results are single-seed point estimates except where stated; only the `policy_conflict` kind-split cell has three 2B seeds (sd 0.03) and two 8B seeds, both v6 checkers are single-seed, and the 8B v4 two-seed gap (test macro 0.956 vs. 0.948) falls on two dimensions, not one. Bootstrap intervals are given only where per-example scores were saved (InjecAgent). The 8B v4 AgentDojo injection cell moved by 0.1 between runs we could not reconcile. Differences smaller than these bands are interpreted nowhere in the paper by intent; where we have failed, the reader should discount them. Kind-split variance over *which* kinds were withheld is not estimated.

**Baselines.** The rule baseline is an uninformative floor on `policy_conflict` and `unauthorized_scope`. The symbolic baseline was run on splits regenerated with the current generator, not the exact v4 files the decoders were scored on, so its cells and the decoders’ are on different draws with the same seed and config. The zero-shot guardrails were prompted through chat-template arguments verified only against test doubles, and that is how the Granite Guardian verdict-position error survived until this revision; an audit found it, and every Guardian number in this paper is from the corrected read. No positive control on a native risk category has been run for either model, though the corrected Guardian reaches AUROC 0.98 on `financial_commitment` and 0.999 AUPRC on AgentDojo’s financial label, so the read is live. `gpt-oss-safeguard` emits a bi-

nary verdict and is scored on 400 rows. The agent-self arm answers one question and is scored on eleven. Baseline rows on the external benchmarks use 1,000-row subsamples with different positive rates from the checker rows.

**External labels are partly ours.** AgentDojo’s policy predicates and InjecAgent’s five policies are self-written; injection and scope labels are the benchmarks’ own. InjecAgent’s policy label is a call-shape judgement and does not bear on identity. AgentDojo’s `unauthorized_scope` counts harmless auxiliary reads as out of scope and has not been adjudicated. The InjecAgent port fills principal and scope with placeholders. The v6 checkers have not been run on the four-suite AgentDojo export. Calibration does not transfer without a few hundred in-domain labels.

**Composition and approval.** §5 is on v2 data and the v2 decoder; the approval-elimination curve’s non-zero values exist only after reweighting a 58%-incident split to an asserted 5% deployment rate, and its denominator is all scored actions, not the review queue (Appendix D). No latency, cost, fail-open behaviour, or adaptive-attack evaluation of the checker is reported. A single English template generator produces every synthetic example.

## ETHICS STATEMENT

All training data are synthetic and contain no personal data; the external benchmarks are used under their published licences and no new human annotation was collected. The base models (Granite 3.3, ModernBERT, granite-embedding-r2) and our code are Apache-2.0. A pre-execution risk checker is dual-use only in the weak sense that any classifier can be tuned to permit what it should refuse; we release policies and thresholds as customer-authored inputs, not fixed defaults, and state plainly that no result here licenses a real-world safety claim. The blind-judge and agent-self experiments used commercial and open models for inference only.

## REFERENCES

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Eliot Wynne, Yarin Gal, and Xander Davies. A benchmark for measuring harmfulness of llm agents. *arXiv preprint arXiv:2410.09024*, 2024.
- Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. Airgapagent: Protecting privacy-conscious conversational agents. *arXiv preprint arXiv:2405.05175*, 2024.
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. Struq: Defending against prompt injection with structured queries. *arXiv preprint arXiv:2402.06363*, 2024a. To appear at USENIX Security Symposium 2025.
- Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. Secalign: Defending against prompt injection with preference optimization. *arXiv preprint arXiv:2410.05451*, 2024b. To appear at ACM CCS 2025.
- Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure AI agents. *arXiv preprint arXiv:2505.03574*, 2025.
- Joseph Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Mike Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Runpta, Emina Torlak, and Andrew Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages*, 8 (OOPSLA2), 2024.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *arXiv preprint arXiv:2406.13352*, 2024.

---

648 Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian,  
649 Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections  
650 by design. *arXiv preprint arXiv:2503.18813*, 2025.

651 Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. Aegis: Online adaptive  
652 ai content safety moderation with ensemble of llm experts. *arXiv preprint arXiv:2404.05993*,  
653 2024.

654 Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural  
655 networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*,  
656 2017.

657 Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kici-  
658 man. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint*  
659 *arXiv:2403.14720*, 2024.

660 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang,  
661 and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint*  
662 *arXiv:2106.09685*, 2021.

663 IBM Granite Team. Granite 3.0 / 3.3 language models: Model card and technical report. Technical  
664 report, IBM Research, 2024. See also Granite 3.3 (2025) release notes.

665 Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael  
666 Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama guard: Llm-  
667 based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*,  
668 2023.

669 Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. The task shield: Enforcing task alignment to  
670 defend against indirect prompt injection in LLM agents. *arXiv preprint arXiv:2412.16682*, 2024.

671 OASIS. extensible access control markup language (xacml) version 3.0. OASIS Standard, <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>,  
672 2013.

673 Open Policy Agent. Open policy agent (opa) documentation. <https://www.openpolicyagent.org/docs/latest/>. Accessed 2026-09-22.

674 OpenAI. gpt-oss-safeguard: Open-weight reasoning models for safety policy classification. Techni-  
675 cal report, OpenAI, 2025. Model and technical report.

676 OWASP GenAI Security Project. OWASP securing agentic applications guide. <https://genai.owasp.org/>, 2025. Accessed 2026-09-22.

677 Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedap-  
678 ati, Pierre Dognin, Keerthiram Murugesan, Erik Miehl, Martin Santillan Qian, Ramya Rama-  
679 murthy, Giulio Zizzo, Amrith Elthouky, Michael Hind, Prasanna Ram, Payel Das, Pin-Yu Chen,  
680 Jerret Cunningham, Ananya Agarwal, Prasanna Sattigeri, Kush Raghavan, and Kush R. Varshney.  
681 Granite guardian. *arXiv preprint arXiv:2412.07724*, 2024.

682 Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer,  
683 Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar,  
684 Christina D. Richards, and Mengzhi Wang. Zanzibar: Google’s consistent, global authorization  
685 system. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019.

686 Pro-GenAI. Agent action guard: Classifying AI agent actions to ensure safety and reliability.  
687 Preprint, <https://doi.org/10.20944/preprints202510.1415.v2>; code <https://github.com/Pro-GenAI/Agent-Action-Guard>, 2025. Accessed 2026-09-22.

688 Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann  
689 Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with  
690 an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.

Table 7: Decoder 2B v2, heldout\_family ( $n=6640$ ) AUPRC per dimension: full, identity-stripped, and rule baseline.

| Dimension                  | Decoder (full) | Decoder (stripped) | Rule baseline |
|----------------------------|----------------|--------------------|---------------|
| unauthorized_scope         | 1.000          | 0.116              | 0.512         |
| policy_conflict            | 0.999          | 0.345              | 0.315         |
| insufficient_context       | 0.849          | 0.418              | 0.235         |
| prompt_injection_influence | 0.810          | 0.810              | 0.301         |
| suspicious_action_sequence | 0.952          | 0.950              | 0.334         |
| financial_commitment       | 1.000          | 1.000              | 1.000         |

Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing ai agents with privilege control. *arXiv preprint arXiv:2504.11703*, 2025.

Benjamin Warner, Antoine Chaffin, Benoît Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. Isolategpt: An execution isolation architecture for llm-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*, 2025. *arXiv preprint arXiv:2403.04960*.

Wenjun Zeng, Yuchen Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahlteiz. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. *arXiv preprint arXiv:2410.02644*, 2025. Accepted at ICLR 2025.

Peter Yong Zhong, Siyuan Chen, Ruiqi Wang, McKenna McCall, Ben L. Titzer, Heather Miller, and Phillip B. Gibbons. Rtbas: Defending LLM agents against prompt injection and privacy leakage. *arXiv preprint arXiv:2502.08966*, 2025.

## A IDENTITY ABLATION: ALL DIMENSIONS, V2

Table 7 gives the v2 per-dimension numbers behind Figure 4; Table 8 gives the v4 numbers behind Figure 1. The v4 ablation replicates the pattern with one change: `privilege_escalation` survives stripping in v4 (1.000 to 1.000) because the v4 renderer puts the deciding `authority_before/authority_after` fields into the tool-call text; in v2 they were never rendered and every arm scored 0.46–0.53 on it, so it is omitted below.

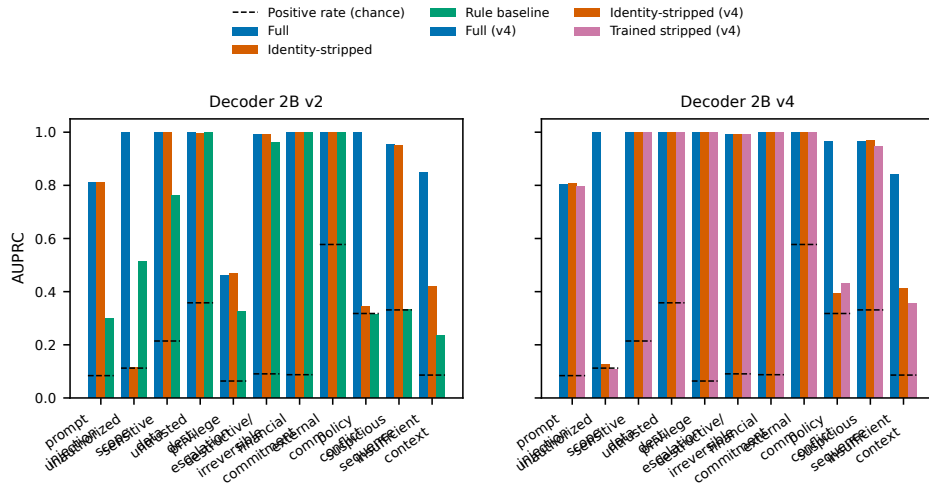


Figure 4: Decoder 2B v2, per-dimension AUPRC on heldout\_family ( $n=6640$ ): full model vs. identity-stripped vs. rule baseline, with each dimension’s positive rate (chance) as a dashed line. The drop is confined to unauthorized\_scope, policy\_conflict, and insufficient\_context.

| Dimension                               | $n$ (pos. rate) | Full  | Eval-stripped | Train-stripped | Rule  |
|-----------------------------------------|-----------------|-------|---------------|----------------|-------|
| unauthorized_scope                      | 6335 (.108)     | 0.999 | 0.125         | 0.107          | 0.507 |
| policy_conflict                         | 1834 (.371)     | 0.964 | 0.392         | 0.429          | 0.370 |
| insufficient_context                    | 6653 (.087)     | 0.841 | 0.412         | 0.357          | 0.232 |
| prompt_injection_influence <sup>†</sup> | 6653 (.082)     | 0.804 | 0.806         | 0.797          | 0.287 |
| suspicious_action_sequence              | 1392 (.317)     | 0.966 | 0.968         | 0.947          | 0.318 |
| privilege_escalation                    | 6653 (.071)     | 1.000 | 1.000         | 1.000          | 0.364 |
| destructive_or_irreversible_action      | 6653 (.088)     | 0.992 | 0.991         | 0.991          | 0.961 |
| sensitive_data_exposure                 | 1558 (.208)     | 1.000 | 1.000         | 1.000          | 0.752 |
| untrusted_destination                   | 2172 (.357)     | 1.000 | 1.000         | 0.998          | 1.000 |
| financial_commitment                    | 6653 (.095)     | 1.000 | 1.000         | 1.000          | 1.000 |
| external_communication                  | 287 (.523)      | 1.000 | 1.000         | 1.000          | 1.000 |
| Macro, 11 dims                          |                 | 0.960 | 0.790         | 0.784          | 0.614 |
| Non-saturated macro, 7 dims             |                 | 0.939 | 0.672         | 0.662          | 0.404 |

Table 8: Numbers behind Figure 1, all eleven dimensions. Decoder 2B v4, heldout\_family, AUPRC with evaluable  $n$  and positive rate: full input, identity removed at evaluation, identity removed at training, and the rule baseline. First three rows are functions of the removed fields. The non-saturated macro averages the seven dimensions on which the rule baseline is below 0.9 (everything except untrusted\_destination, financial\_commitment, external\_communication, destructive\_or\_irreversible\_action). <sup>†</sup>v4 text carries the injection label leak (§7).

Table 9: Zero-shot safety models per dimension on `heldout_family`: Granite Guardian 3.3 8B (yes/no logits,  $n=2000$  subsample) AUPRC and AUROC, and gpt-oss-safeguard 20B (binary verdict,  $n=400$  subsample) AUPRC, full context. Positive rates are each subsample’s. Guardian is scored from the corrected verdict-token position (§3).

| Dimension                                       | pos. rate | Guardian AUPRC | Guardian AUROC | gpt-oss AUPRC |
|-------------------------------------------------|-----------|----------------|----------------|---------------|
| <code>prompt_injection_influence</code>         | 0.085     | 0.708          | 0.858          | 0.147         |
| <code>unauthorized_scope</code>                 | 0.112     | 0.204          | 0.736          | 0.135         |
| <code>sensitive_data_exposure</code>            | 0.210     | 0.306          | 0.693          | 0.282         |
| <code>untrusted_destination</code>              | 0.363     | 0.665          | 0.855          | 0.364         |
| <code>privilege_escalation</code>               | 0.065     | 0.144          | 0.759          | 0.092         |
| <code>destructive_or_irreversible_action</code> | 0.093     | 0.227          | 0.701          | 0.149         |
| <code>financial_commitment</code>               | 0.087     | 0.675          | 0.979          | 0.098         |
| <code>external_communication</code>             | 0.639     | 0.905          | 0.876          | 0.775         |
| <code>policy_conflict</code>                    | 0.311     | 0.437          | 0.689          | 0.495         |
| <code>suspicious_action_sequence</code>         | 0.317     | 0.365          | 0.557          | 0.408         |
| <code>insufficient_context</code>               | 0.084     | 0.088          | 0.526          | 0.099         |

---

## B POLICY GENERALIZATION: MACRO AUPRC AND PER-SEED VALUES

The v4 render fix also confirms that `privilege_escalation`’s earlier AUROC-high/AUPRC-low signature was a data defect, not a model limit: rendering the deciding fields lifts it from 0.505 to 1.000 AUPRC on `heldout_family`.

## C MULTI-POLICY STACKING: FULL TABLE AND CURVES

Table 11 gives the numbers behind Figure 2 plus the `joint` strategy (rules from all  $k$  policies merged into one bundle and evaluated once in threshold mode), review rate, covered-benign count, and the mean risk-weighted cost the expected-cost rule optimises. The stack order is `permissive`, `balanced`, `conservative`, `expected-cost-example`, then eleven single-dimension synthetic policies in enum order ( $k = 5..15$ ); the synthetic policies change no FPR value. The covered-benign denominator changes from 3,785 at  $k=1$  to 2,790 at  $k \geq 2$ , so the  $k=1$  row is on a different population from the rest. All rows are the v2 decoder and the v2 encoder on v2 data.

## D APPROVAL ELIMINATION

The product-facing quantity is how many actions can be auto-allowed at a chosen incident rate: sort examples by the decoder’s expected cost of `ALLOW` and take the longest prefix whose realised incident rate (fraction of auto-allowed actions carrying a `yes` on a high-severity dimension) stays under a budget. This is a probabilistic authorization decision; it is not the “review pricing” posture of §2, and a deployment adopting it takes on that audit burden. The denominator below is *all scored actions*, not the subset a bundle would otherwise route to review, so the numbers are “fraction of traffic auto-allowed,” not “fraction of the review queue removed”; the two differ by the baseline review rate. Because `heldout_family` is adversarial-heavy (58% base incident rate), Table 12 reports both the raw curve and a curve importance-reweighted to an *assumed 5%* deployment incident rate (weights 0.086 on risky and 2.26 on benign rows), which is valid only if deployment traffic has the same class-conditional score distributions as this split. Elimination is computed with a 95% Wilson upper bound (Wilson, 1927) on the realised incident rate in the auto-allowed set, applied to weighted counts, which understates the interval’s width.

Under the 5% assumption the calibrated decoder auto-allows 88.8% of scored actions at a 1% incident budget and 99.2% at 5%. The unweighted column is zero at every budget because the split’s top-ranked rows already contain risky examples by construction. The result is therefore a synthetic ranking curve multiplied by an assumed prior; a sweep over the assumed rate, the rule baseline’s and encoder’s curves under the same procedure, and the `insufficient_context`-excluded variant (§6) have not been run.

## E AGENT SELF-JUDGMENT: ALL DIMENSIONS

A cheaper alternative to an external checker is to ask the acting agent, inside its own system prompt, whether to proceed. We give the untrained 8B instruct model an agent system prompt carrying the identical entitlements, delegated scope, and policy text, then the pending call, and read one `ALLOW/STOP` verdict off the first output token, scored against all eleven labels on 2,000 `heldout_family` rows (Table 13, Figure 8). On the generator’s entitlement-based scope labels the agent is at chance (AUROC 0.54–0.67 on the three identity dimensions); the checker beats it on all eleven. One verdict scored against eleven labels is structurally limited, so part of the gap is definitional, and the finding does not travel: on `InjecAgent` (§7) the same prompt returns `STOP` on the attacker-tool calls and `ALLOW` on the user-task calls, which is exactly that benchmark’s scope label (AUPRC 1.000). The agent recognises an action nobody asked for; it does not check a call against a delegated grant. The injection reversal (0.166 to 0.507 with the preamble removed) is on leaked v4 text and is reported as confounded.

The checker beats the agent’s single-verdict score on all eleven dimensions by 0.45–0.90 AUPRC. The small identity signal that does exist in the agent (`unauthorized_scope` AUROC 0.61 full vs. 0.48 stripped) shows it reads the context a little without turning it into a usable decision. The 2B agent-self condition behaves the same way and is omitted.

Table 10: Decoder 2B, macro AUPRC and policy\_conflict (pc) AUPRC across splits for the v3 (15 policy kinds, 2 withheld), v4 (same data, privilege\_escalation render fix), and v5 (30 kinds, 4 withheld) generators, seed 0; last column an 8B LoRA on v4 data, seed 0. Three-seed values for the heldout\_policy\_kind pc cell: v4 0.752, 0.689, 0.711; v5 0.704, 0.755, 0.717; 8B v4 (two seeds) 0.851, 0.825. The kind-split cells below are seed 0, so they differ from the seed means in Table 4. Rule baseline policy\_conflict on the kind split: 0.405 (v3/v4), 0.325 (v5), equal to the positive rate.

| Split                   | v3 macro | v4 macro | v5 macro | v3 pc | v4 pc | v5 pc | 8B v4 pc |
|-------------------------|----------|----------|----------|-------|-------|-------|----------|
| test                    | 0.908    | 0.960    | 0.957    | 0.994 | 0.994 | 0.996 | 0.990    |
| heldout_family          | 0.914    | 0.960    | 0.951    | 0.941 | 0.964 | 0.933 | 0.965    |
| heldout_policy_kind     | 0.889    | 0.944    | 0.941    | 0.653 | 0.752 | 0.704 | 0.851    |
| heldout_policy_phrasing | 0.921    | 0.968    | 0.970    | 0.987 | 0.993 | 0.980 | 0.978    |

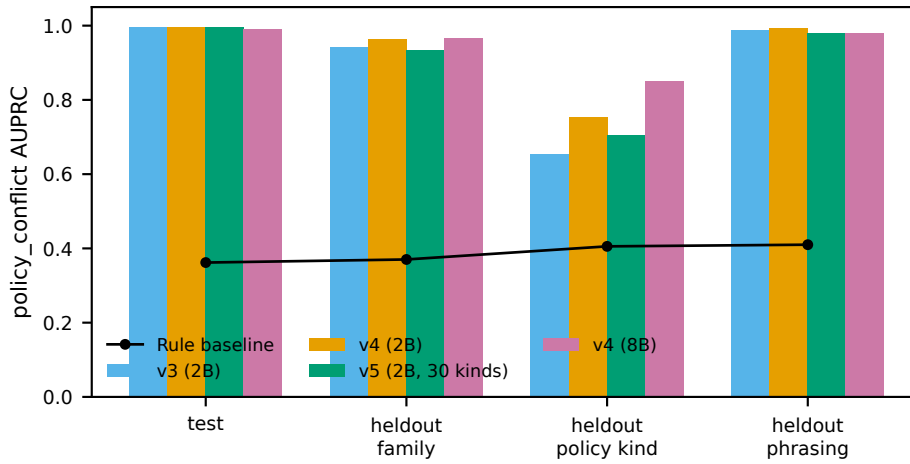


Figure 5: Decoder 2B, policy\_conflict AUPRC across splits for the v3, v4, and v5 generators, plus the 8B LoRA on v4 data, with the rule baseline’s policy\_conflict AUPRC on each split shown as a line. Bars for test, heldout\_family, and heldout\_policy\_phrasing sit near the ceiling; heldout\_policy\_kind drops toward, though still above, the rule baseline, for every generator and both sizes.

## F BLIND JUDGE: FULL AGREEMENT TABLE

## G EXTERNAL BENCHMARKS: FULL TABLES

The pair test above compares poisoned against plain clean only, which is shorter in every group; Table 19 adds two more controls, padded and instruction, that refill the same slot with benign text byte-matched to the poisoned observation’s length, and reports the full win count against all three.

**Per-example logits.** The 2B v4 injection head never fires on AgentDojo: raw logits for all 4,133 evaluable calls lie between  $-5.0$  and  $-3.0$ , the highest-scoring positive is  $-3.25$ , and no call scores above zero, while on the same rows its policy\_conflict head has median raw logit  $+2.5$  for positives and  $-3.5$  for negatives. The 8B v4 head is live but shifted (median  $-1.75$  for positives vs.  $-3.50$  for negatives, 29% of positives above zero vs. 0.7% of negatives). After the v6 retrain, 2B v6 has median  $+4.5$  vs.  $-3.0$  (92% vs. 23% above zero) and 8B v6  $+6.25$  vs.  $-3.25$  (94% vs. 8%). On InjecAgent the v4 heads sit at raw logit  $-4$  on poisoned and clean alike; the v6 heads have median raw  $+3.5$  vs.  $-6.25$  (2B) and  $+2.25$  vs.  $-3.5$  (8B), and 58% of poisoned calls score above 0 at both sizes against 9% (2B) and 0.1% (8B) of clean. Inference is deterministic (per-example raw difference 0 between repeated runs).

Table 11: Multi-policy stacking, v2 heldout\_family ( $n=6640$ ). FPR counts REVIEW as a false positive.

| Arm                | $k$ | Strategy            | FPR          | FNR   | Review | Benign cov. | Cost  |
|--------------------|-----|---------------------|--------------|-------|--------|-------------|-------|
| Decoder 2B         | 1   | independent / joint | 0.015        | 0.253 | 0.31   | 3785        | 1.038 |
| Decoder 2B         | 1   | expected_cost       | 0.058        | 0.170 | 0.37   | 3785        | 0.805 |
| Decoder 2B         | 2   | independent         | 0.140        | 0.080 | 0.30   | 2790        | 0.194 |
| Decoder 2B         | 2   | joint               | 0.131        | 0.105 | 0.28   | 2790        | 0.256 |
| Decoder 2B         | 2   | expected_cost       | 0.082        | 0.070 | 0.28   | 2790        | 0.125 |
| Decoder 2B         | 3   | independent         | <b>0.966</b> | 0.016 | 0.66   | 2790        | 0.202 |
| Decoder 2B         | 3   | joint               | 0.905        | 0.051 | 0.63   | 2790        | 0.290 |
| Decoder 2B         | 3   | expected_cost       | 0.082        | 0.070 | 0.28   | 2790        | 0.125 |
| Decoder 2B         | 8   | independent         | 0.966        | 0.007 | 0.58   | 2790        | 0.139 |
| Decoder 2B         | 8   | expected_cost       | 0.082        | 0.070 | 0.28   | 2790        | 0.125 |
| Decoder 2B         | 15  | independent         | 0.967        | 0.003 | 0.40   | 2790        | 0.114 |
| Decoder 2B         | 15  | expected_cost       | 0.082        | 0.070 | 0.28   | 2790        | 0.125 |
| Encoder granite-r2 | 1   | independent / joint | 0.346        | 0.163 | 0.53   | 3785        | 0.790 |
| Encoder granite-r2 | 1   | expected_cost       | 0.589        | 0.050 | 0.71   | 3785        | 0.449 |
| Encoder granite-r2 | 2   | independent         | 0.710        | 0.042 | 0.26   | 2790        | 0.159 |
| Encoder granite-r2 | 2   | expected_cost       | 0.921        | 0.010 | 0.37   | 2790        | 0.107 |
| Encoder granite-r2 | 3   | independent         | 0.990        | 0.008 | 0.21   | 2790        | –     |
| Encoder granite-r2 | 3   | expected_cost       | 0.921        | 0.010 | 0.37   | 2790        | –     |

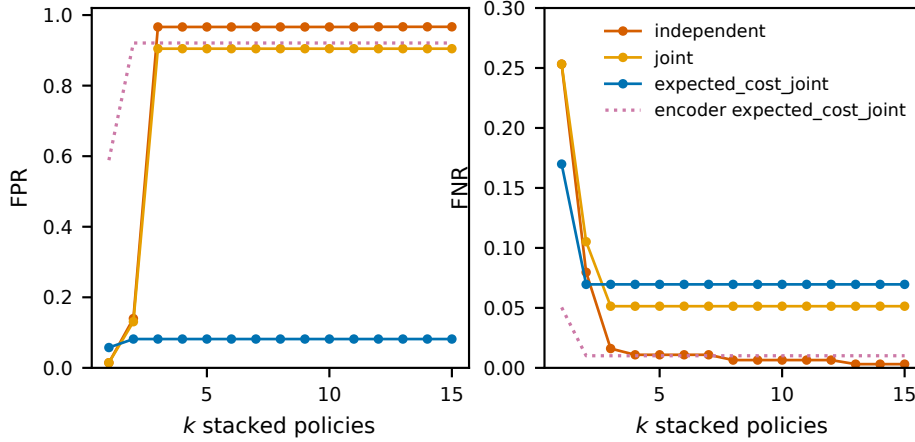


Figure 6: Decoder 2B, FPR (left) and FNR (right) vs. number of stacked policies  $k$  on heldout\_family ( $n=6640$ ), for independent, joint, and expected\_cost\_joint strategies; the granite-embedding-r2 encoder’s expected\_cost\_joint curve is dotted. The independent step at  $k=3$  is the fail-closed conservative bundle entering; the flat expected\_cost lines follow from the covered set saturating at  $k=2$  (§5).

**Recalibration.** Refitting per-dimension isotonic calibrators on a random half of the AgentDojo banking calls and testing on the other half (five seeds) gives macro ECE 0.008 (2B) and 0.010 (8B) with AUPRC unchanged; the 8B unauthorized\_scope head goes from ECE 0.60 to 0.01 by learning to output the base rate. Most of the gain is reached with 250–500 labelled calls; the sweep is 8B only, isotonic only, on two of the five dimensions, and Platt refits are much worse (macro ECE 0.030 / 0.048).

**Other suites.** unauthorized\_scope outside banking is carried by the rule baseline (0.749 on slack, 0.733 on workspace, where nearly half of all calls are labelled out of scope); 2B adds under 0.09 and both 8B arms are below the rule. policy\_conflict on slack, travel, and workspace uses one generic policy per suite at 0.7–8% base rates: 2B AUROC is 0.78–0.89 but AUPRC 0.005–0.49, so we claim policy transfer on banking and InjecAgent only.

Table 12: Decoder 2B, approval elimination on `heldout_family` (v2  $n=6640$ , v4  $n=6653$ ), bundle balanced. “raw” is the split’s own 57–58% base incident rate; “rw” reweights rows so the incident rate is 5%. Zeros below a 1% budget mean the Wilson bound is unreachable at this sample size, not that nothing can be allowed.

| Budget | Elim. (v2, raw) | Elim. (v2, rw) | Elim. (v4, rw) | Review (v4, rw) |
|--------|-----------------|----------------|----------------|-----------------|
| 0.1%   | 0.000           | 0.000          | 0.000          | 0.969           |
| 0.5%   | 0.000           | 0.000          | 0.000          | 0.969           |
| 1%     | 0.000           | 0.888          | 0.903          | 0.067           |
| 2%     | 0.000           | 0.961          | 0.952          | 0.021           |
| 5%     | 0.000           | 0.992          | 0.992          | 0.001           |

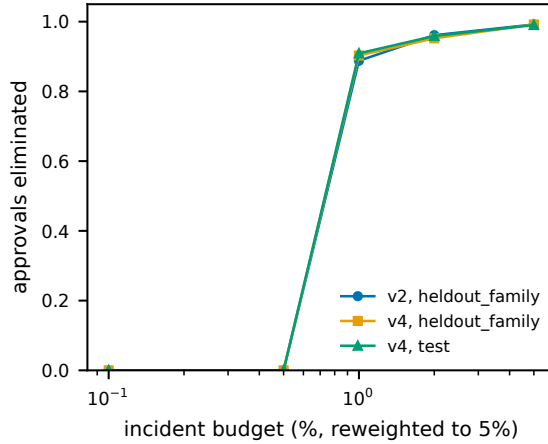


Figure 7: Approvals eliminated vs. incident budget, reweighted to a 5% deployment incident rate, decoder 2B v2 and v4 on `heldout_family` and v4 on test, log-scaled budget axis. The three curves nearly overlap.

## H INJECTION LABEL LEAK: DIAGNOSIS AND FIX

After the 2B checker failed on AgentDojo injection we contrasted the rendered call between injected and non-injected synthetic examples. The v1–v5 generator wrote an `instructed_target` argument into every injected call and into no other call (presence 1.00 vs. 0.00 over 13,312 v4 rows), put a `to` argument into 100% of injected calls vs. 3% of others, and drew the injected target from five literals, three containing “attacker” (`attacker-agent-007`, `external-support@attacker.example`, `shadow-ops@attacker.example`). Injected observations were five short templates (median 132 characters) against AgentDojo’s 2,000-character record dumps. The v6 renderer (2.0.0) removes `instructed_target`, chooses the target argument key from the tool kind and fills it for benign calls too, draws targets from pools shared between injected and benign calls, and renders 300–2,000 character multi-format observations with the instruction at a random position. A leakage check on argument-key presence parity and a token denylist now runs before training; such a check would have caught this leak from the synthetic data alone, so the accurate statement is that no synthetic *held-out split* could reveal it, not that nothing synthetic could. Why the synthetic metric did not move: the generator draws the injection label from two origins, an injected instruction in untrusted content (probability 0.06 when untrusted content is present) and a compromised tool schema (0.02, with no untrusted text rendered), so about a quarter of positives carry no injected text. A detector reading only the observation and a detector reading only the leaked key are both capped near 0.75–0.80 AUPRC on this label; the v4 and v6 checkers score 0.80 and 0.79. An earlier draft stated that a key-presence detector would score 1.0; that was wrong. Judge agreement on `prompt_injection_influence` ( $\kappa=0.905$ , Table 14) was measured on v4 text and has not been re-run on v6. An AUPRC split by label origin would show what the head reads and has not been computed.

| Dimension (pos. rate)                       | Agent | Stripped | Checker |
|---------------------------------------------|-------|----------|---------|
| unauthorized_scope (0.106)                  | 0.135 | 0.101    | 0.999   |
| policy_conflict (0.385)                     | 0.509 | 0.419    | 0.964   |
| insufficient_context (0.079)                | 0.086 | 0.083    | 0.841   |
| privilege_escalation (0.075)                | 0.103 | 0.101    | 1.000   |
| prompt_injection_infl. <sup>†</sup> (0.075) | 0.166 | 0.507    | 0.804   |
| Macro (11 dims)                             | 0.242 | 0.281    | 0.960   |

Table 13: Agent self-judgment (8B, untrained, one ALLOW/STOP verdict), with and without identity context, vs. the trained 2B v4 checker, AUPRC on the same 2,000 heldout\_family rows. <sup>†</sup>Leaked v4 text.

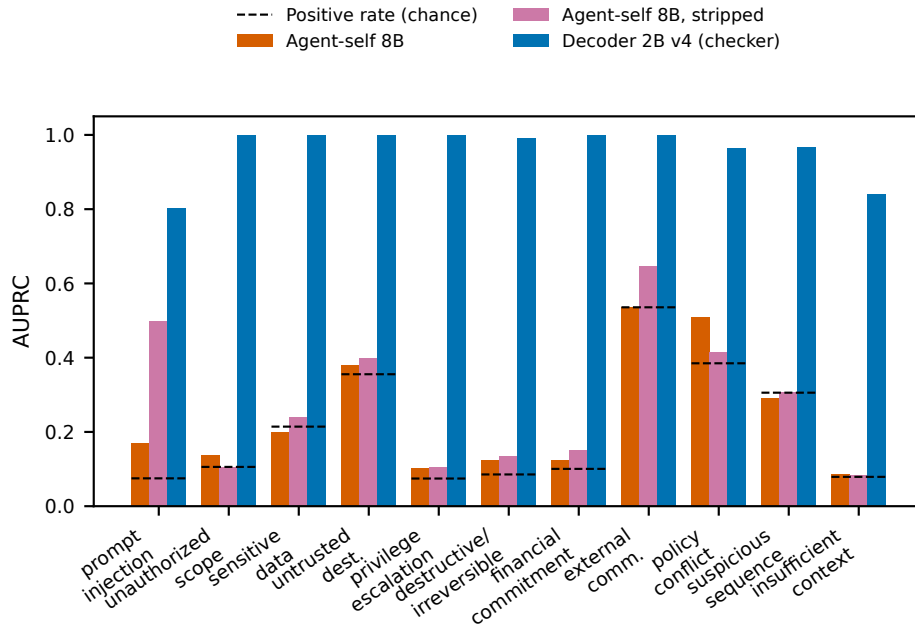


Figure 8: Per-dimension AUPRC on heldout\_family ( $n=2000$  for the agent-self-8B rows,  $n=6653$  for the decoder), positive rate as a dashed line. The agent’s bars track chance everywhere except policy\_conflict and external\_communication; the external checker clears 0.84 on every dimension.

Table 14: Blind LLM-judge agreement with the generator label, yes vs. not-yes,  $n=600$  v4 rows (gpt-5.6-sol, blind to the generator’s latent scenario, one prompt, definitions copied from the generator’s docstrings), full and identity-stripped renderings. Agreement/recall/precision are for the full-context pass. No intervals; the injection row is on leaked text.

| Dimension                          | $\kappa$ full | $\kappa$ stripped | Agreement | Judge recall | Judge precision |
|------------------------------------|---------------|-------------------|-----------|--------------|-----------------|
| privilege_escalation               | 0.909         | 0.909             | 0.988     | 1.00         | 0.84            |
| prompt_injection_influence         | 0.905         | 0.905             | 0.983     | 0.84         | 1.00            |
| policy_conflict                    | 0.714         | 0.029             | 0.862     | 0.89         | 0.78            |
| destructive_or_irreversible_action | 0.511         | 0.542             | 0.897     | 0.60         | 0.54            |
| suspicious_action_sequence         | 0.404         | 0.428             | 0.852     | 0.82         | 0.33            |
| unauthorized_scope                 | 0.391         | 0.040             | 0.800     | 0.74         | 0.38            |
| financial_commitment               | 0.359         | 0.300             | 0.935     | 0.24         | 0.92            |
| external_communication             | 0.331         | 0.361             | 0.897     | 0.86         | 0.23            |
| untrusted_destination              | 0.213         | 0.205             | 0.777     | 0.41         | 0.29            |
| insufficient_context               | 0.059         | -0.036            | 0.843     | 0.14         | 0.15            |
| sensitive_data_exposure            | 0.002         | 0.061             | 0.783     | 0.17         | 0.08            |

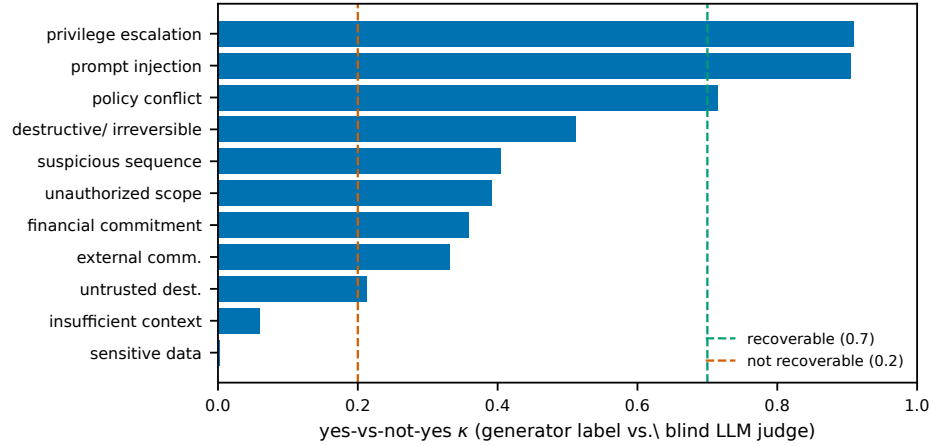


Figure 9: Yes-vs-not-yes  $\kappa$  per dimension, sorted, with the recoverable (0.7) and not-recoverable (0.2) tier lines.

Table 15: AUPRC against the judge’s labels on the judged `heldout_family` rows ( $n=257$ ), decoder 2B v4 vs. rule baseline, with the decoder’s AUPRC against the generator’s own labels on the full split for reference. The lookup dimensions collapse for both arms because the judge defines them differently (e.g. `financial_commitment` requires funds to move; `sensitive_data_exposure` requires a lower-sensitivity destination), so they are not evidence for or against either arm.

| Dimension                                       | Decoder 2B v4 | Rule baseline | Decoder vs. generator labels |
|-------------------------------------------------|---------------|---------------|------------------------------|
| <code>prompt_injection_influence</code>         | <b>1.000</b>  | 0.459         | 0.804                        |
| <code>privilege_escalation</code>               | <b>0.977</b>  | 0.297         | 1.000                        |
| <code>policy_conflict</code>                    | <b>0.889</b>  | 0.502         | 0.964                        |
| <code>untrusted_destination</code>              | <b>0.864</b>  | 0.785         | 1.000                        |
| <code>destructive_or_irreversible_action</code> | 0.635         | 0.509         | 0.992                        |
| <code>unauthorized_scope</code>                 | 0.592         | 0.509         | 0.999                        |
| <code>suspicious_action_sequence</code>         | 0.528         | 0.467         | 0.966                        |
| <code>external_communication</code>             | 0.437         | 0.381         | 1.000                        |
| <code>sensitive_data_exposure</code>            | 0.405         | 0.291         | 1.000                        |
| <code>financial_commitment</code>               | 0.327         | 0.280         | 1.000                        |
| <code>insufficient_context</code>               | 0.090         | 0.090         | 0.841                        |
| macro, 11 dims                                  | 0.613         | 0.415         | 0.960                        |
| macro, 10 dims (injection excluded)             | 0.574         | 0.411         | 0.976                        |

Table 16: AUPRC on AgentDojo banking traces, zero-shot,  $n=4,215$  ( $n=4,133$  for injection). `untrusted_destination` has positives only and is excluded from macro. Agent-self is on a fixed 1,000-row subsample whose positive rates differ from the full set’s (`unauthorized_scope` 0.166 vs. 0.253); the corrected Guardian row is on an independently drawn 1,000-row subsample (`unauthorized_scope` positive rate 0.269). Macro ECE of the calibrated arms is 0.20–0.27 here against under 0.03 on synthetic data; the eval-stripped rows are raw margins without a calibrator. Macro here is over five dimensions.

| Arm                        | injection | scope | policy | financial | destructive | macro |
|----------------------------|-----------|-------|--------|-----------|-------------|-------|
| Rule baseline              | 0.250     | 0.253 | 0.237  | 0.896     | 1.000       | 0.527 |
| Decoder 2B v4              | 0.128     | 0.257 | 0.849  | 1.000     | 1.000       | 0.647 |
| 2B v4, eval-stripped       | 0.125     | 0.218 | 0.694  | 1.000     | 1.000       | 0.608 |
| 2B v4, train-stripped      | 0.150     | 0.234 | 0.390  | 1.000     | 1.000       | 0.555 |
| Decoder 8B v4              | 0.701     | 0.355 | 0.626  | 0.923     | 0.999       | 0.721 |
| 8B v4, eval-stripped       | 0.743     | 0.266 | 0.404  | 0.934     | 1.000       | 0.669 |
| Decoder 2B v6 (leak-free)  | 0.693     | 0.214 | 0.670  | 0.935     | 1.000       | 0.702 |
| 2B v6, eval-stripped       | 0.669     | 0.306 | 0.521  | 0.941     | 1.000       | 0.687 |
| Decoder 8B v6 (leak-free)  | 0.838     | 0.239 | 0.785  | 0.945     | 0.831       | 0.728 |
| 8B v6, eval-stripped       | 0.819     | 0.354 | 0.450  | 0.979     | 0.975       | 0.715 |
| Granite Guardian 3.3 8B    | 0.372     | 0.539 | 0.587  | 0.999     | 0.111       | 0.521 |
| Agent-self 8B (ALLOW/STOP) | 0.291     | 0.442 | 0.449  | 0.591     | 0.105       | 0.375 |

Table 17: AUPRC on InjecAgent, zero-shot,  $n=4,250$  ( $n=3,196$  for injection, positive rate 0.500). Pair win: fraction of the 1,598 poisoned/clean pairs on which the poisoned variant scores higher on injection; tie: fraction with identical raw score. Baseline rows on a fixed 1,000-row subsample (injection  $n=767$ , positive rate 0.471). Macro is over five dimensions.

| Arm                        | injection | scope | policy | financial | destructive | macro | pair win | tie   |
|----------------------------|-----------|-------|--------|-----------|-------------|-------|----------|-------|
| Rule baseline              | 0.500     | 0.752 | 0.464  | 0.683     | 1.000       | 0.680 | –        | –     |
| Decoder 2B v4              | 0.549     | 0.732 | 0.965  | 1.000     | 1.000       | 0.849 | 0.504    | 0.239 |
| Decoder 8B v4              | 0.544     | 0.849 | 0.977  | 0.734     | 1.000       | 0.821 | 0.353    | 0.385 |
| 8B v4, eval-stripped       | 0.556     | 0.843 | 0.968  | 0.766     | 1.000       | 0.827 | 0.382    | 0.397 |
| Decoder 2B v6 (leak-free)  | 0.878     | 0.704 | 0.922  | 0.970     | 0.998       | 0.894 | 0.977    | 0.005 |
| Decoder 8B v6 (leak-free)  | 0.943     | 0.677 | 0.965  | 0.771     | 0.997       | 0.870 | 0.977    | 0.012 |
| 8B v6, eval-stripped       | 0.953     | 0.917 | 0.963  | 0.839     | 0.998       | 0.934 | 0.978    | 0.016 |
| Granite Guardian 3.3 8B    | 0.539     | 0.999 | 0.722  | 0.940     | 0.445       | 0.729 | –        | –     |
| Agent-self 8B (ALLOW/STOP) | 0.530     | 1.000 | 0.647  | 0.061     | 0.178       | 0.483 | –        | –     |

Table 18: InjecAgent pair audit over the 1,598 poisoned/clean pairs: wins, ties, and losses of the poisoned variant on the injection raw margin; two-sided sign test on decided pairs; strict win on the 972 pairs whose destination-trust field is equal in both members; and 95% bootstrap intervals (2,000 resamples over examples) for AUPRC on the three headline dimensions. In every pair the poisoned observation is longer than the clean one.

| Arm             | W    | T   | L   | $p$                 | win, trust eq. | injection CI   | scope CI       | policy CI      |
|-----------------|------|-----|-----|---------------------|----------------|----------------|----------------|----------------|
| 2B v4           | 806  | 382 | 410 | $3 \times 10^{-30}$ | 0.638          | [0.527, 0.572] | [0.714, 0.751] | [0.959, 0.971] |
| 8B v4           | 564  | 616 | 418 | $4 \times 10^{-6}$  | 0.307          | [0.520, 0.566] | [0.835, 0.863] | [0.973, 0.981] |
| 8B v4, stripped | 611  | 634 | 353 | $8 \times 10^{-17}$ | 0.360          | [0.534, 0.579] | [0.831, 0.856] | [0.963, 0.973] |
| 2B v6           | 1562 | 8   | 28  | $< 10^{-300}$       | 0.963          | [0.866, 0.890] | [0.685, 0.722] | [0.913, 0.931] |
| 8B v6           | 1562 | 19  | 17  | $< 10^{-300}$       | 0.970          | [0.936, 0.950] | [0.660, 0.695] | [0.960, 0.970] |
| 8B v6, stripped | 1563 | 25  | 10  | $< 10^{-300}$       | 0.972          | [0.947, 0.959] | [0.910, 0.925] | [0.958, 0.968] |

Table 19: InjecAgent length-matched controls, over the 1,598 attacker-tool groups. Each group has four variants of the same call: poisoned (attacker instruction in the tool response), clean (instruction removed), clean\_padded (benign filler byte-matched to the poisoned length), and clean\_instruction (a benign imperative instruction byte-matched in length). Poisoned is longer than clean in 100% of groups but longer than clean\_padded or clean\_instruction in only 4.3%; destination trust differs between poisoned and clean in 39.2% of groups. W/T/L: poisoned wins/ties/loses against the named control on the injection raw margin; strict win =  $W/(W+T+L)$ ; tie-adjusted win =  $(W+T/2)/(W+T+L)$ ;  $p$  is a two-sided sign test on decided pairs; the last column restricts to the 972 groups where destination trust is equal between poisoned and the control.

| Checker         | Control     | W    | T   | L   | strict win | tie-adj. win | $p$                  | win, trust eq. ( $n=972$ ) |
|-----------------|-------------|------|-----|-----|------------|--------------|----------------------|----------------------------|
| 2B v6           | clean       | 1562 | 8   | 28  | 0.977      | 0.980        | $< 10^{-300}$        | 0.963                      |
| 2B v6           | padded      | 1572 | 6   | 20  | 0.984      | 0.986        | $< 10^{-300}$        | 0.973                      |
| 2B v6           | instruction | 1579 | 3   | 16  | 0.988      | 0.989        | $< 10^{-300}$        | 0.986                      |
| 8B v6           | clean       | 1562 | 19  | 17  | 0.977      | 0.983        | $< 10^{-300}$        | 0.970                      |
| 8B v6           | padded      | 1559 | 17  | 22  | 0.976      | 0.981        | $< 10^{-300}$        | 0.966                      |
| 8B v6           | instruction | 1579 | 16  | 3   | 0.988      | 0.993        | $< 10^{-300}$        | 0.988                      |
| 8B v6, stripped | clean       | 1563 | 25  | 10  | 0.978      | 0.986        | $< 10^{-300}$        | 0.972                      |
| 8B v6, stripped | padded      | 1566 | 20  | 12  | 0.980      | 0.986        | $< 10^{-300}$        | 0.975                      |
| 8B v6, stripped | instruction | 1581 | 13  | 4   | 0.989      | 0.993        | $< 10^{-300}$        | 0.989                      |
| 2B v4           | clean       | 806  | 382 | 410 | 0.504      | 0.624        | $3 \times 10^{-30}$  | 0.638                      |
| 2B v4           | padded      | 497  | 648 | 453 | 0.311      | 0.514        | $1.6 \times 10^{-1}$ | 0.410                      |
| 2B v4           | instruction | 465  | 715 | 418 | 0.291      | 0.515        | $1.2 \times 10^{-1}$ | 0.387                      |

Table 20: AgentDojo, all four suites (30,143 proposed calls: banking 4,215, slack 8,246, travel 10,701, workspace 6,981), prompt\_injection\_influence AUPRC / AUROC per suite, positive rate in the header. v4 checkers only; the v6 checkers have not been run on this export. 2B v4’s below-0.5 AUROC on banking and workspace is systematic, not noise: calls preceded by a tool result score about a quarter-logit lower on this head.

| Arm                  | banking (0.150) | slack (0.186) | travel (0.025) | workspace (0.077) |
|----------------------|-----------------|---------------|----------------|-------------------|
| Rule baseline        | 0.250 / 0.737   | 0.222 / 0.601 | 0.021 / 0.360  | 0.138 / 0.740     |
| Decoder 2B v4        | 0.128 / 0.348   | 0.178 / 0.479 | 0.048 / 0.686  | 0.065 / 0.353     |
| Decoder 8B v4        | 0.701 / 0.895   | 0.433 / 0.738 | 0.131 / 0.834  | 0.752 / 0.941     |
| 8B v4, eval-stripped | 0.743 / 0.915   | 0.421 / 0.726 | 0.146 / 0.844  | 0.727 / 0.943     |
| Agent-self 8B        | 0.354 / 0.832   | 0.292 / 0.687 | 0.026 / 0.484  | 0.246 / 0.866     |

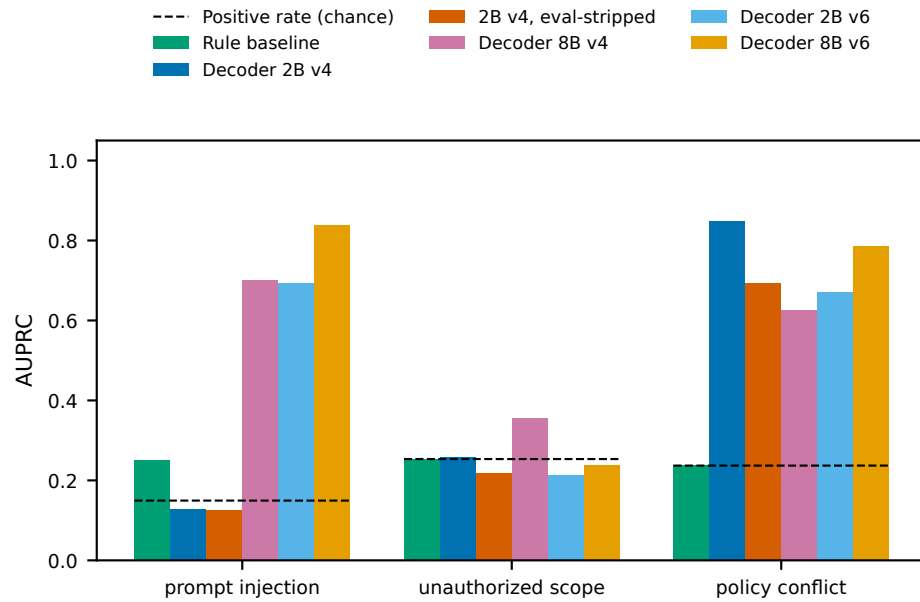


Figure 10: AUPRC on AgentDojo banking traces for the three learned dimensions: rule baseline, decoder 2B v4, 2B v4 eval-stripped, decoder 8B v4, and the leak-free 2B v6 and 8B v6, zero-shot, with each dimension’s positive rate as a dashed line.

---

## I EXPERIMENTAL DETAILS

**Data.** Each version is generated with seed 424242, 3,863 scenarios per tool family over twelve families plus 130 contrastive pairs per axis over fourteen axes, 49,996 rows in total. Rows are routed to splits by a salted hash of the scenario: 60/10/10/10/5% to `train`, `calibration`, `dev`, `test`, and `adversarial`; 12% of tool families are withheld into `heldout_family` before that routing, and scenarios whose policy clause uses a withheld predicate kind (2 of 15 in v4, 4 of 30 in v5) go to `heldout_policy_kind`. Contrastive pairs use an evaluation-heavy ratio table so that every axis reaches each evaluation split. The rendering contract is fixed per version (label schema 1.0).

**Decoders.** `granite-3.3-2b-instruct` and `granite-3.3-8b-instruct`, bf16, LoRA (Hu et al., 2021) with  $r=16$ ,  $\alpha=32$ , dropout 0.05. 2B adapts the attention projections only; lr  $2 \times 10^{-4}$ , three epochs, micro-batch 16, prompts up to 4,096 tokens. 8B adapts attention and MLP projections; lr  $5 \times 10^{-5}$ , one epoch, micro-batch 8, prompts up to 8,192 tokens, chat template on. Both use cosine decay with 3% warmup, weight decay 0.01, gradient clipping 1.0, seed 0 unless a seed is stated, evaluation on dev every 200 steps with early stopping (patience 5). Each dimension is scored from the logit difference of the candidate tokens for *yes* and *no* at the dimension’s answer slot, with a shared prefill across the eleven slots. Temperature scaling is fitted per dimension on `calibration`. All runs use one B200 GPU.

**Encoders.** ModernBERT-large (Warner et al., 2024) and `granite-embedding-r2` with mean pooling and an eleven-way sigmoid head; backbone lr  $2 \times 10^{-5}$ , head lr  $10^{-3}$ , three epochs, micro-batch 32.

**Zero-shot arms.** Granite Guardian 3.3 8B and `gpt-oss-safeguard` 20B receive the policy text as a custom criterion together with the same rendered context; Guardian is scored from the verdict-token logits, `gpt-oss` from its generated verdict. Guardian emits `<score>` before its *yes/no* token; an earlier read took the logits one position before that token, and this revision corrects it to read the token after `<score>`. The agent self-judgment arm is the untrained 8B `instruct` model with an agent system prompt, scored from the `ALLOW/STOP` logits. The blind judge is `gpt-5.6-sol` at low reasoning effort with one prompt containing the eleven docstring definitions.